Third-party libraries and tools

Everything DeployHub is built with, what each piece does and why it was chosen · versions as installed on 26 October 2026

In short: DeployHub is a **Spring Boot** (Java 21) backend and an **Angular** frontend, storing its data in **Oracle Database Free** and running apps with **Docker**. The backend names 16 libraries directly (186 in total, counting what they bring along); the frontend names 12 packages (about 480 in total). Everything is **free and open source** or free to use. Nothing is paid.

1. The big picture

Layer	Main building blocks
Backend (API, workers)	Java 21, Spring Boot 4.1 (web, security, data JPA, validation, actuator, mail), Hibernate, Flyway, Micrometer, springdoc
Frontend (web pages)	Angular 21, PrimeNG 21 (UI components), RxJS, TypeScript
Databases	Oracle Database Free (DeployHub's own data), MySQL 8.4 (apps' one-click databases), H2 (development and tests)
Running apps	Docker (builds and containers), nginx (fixed project addresses), Cloudflare cloudflared (public links), git (fetching code)
Testing	JUnit 6, Mockito, AssertJ, Spring's MockMvc, Testcontainers (real Oracle), Vitest + jsdom (frontend)

2. Backend libraries (Java, managed by Maven)

Versions are chosen by Spring Boot's dependency management, so they are tested together; only springdoc is pinned by DeployHub itself. Declared in `deployhub-backend/pom.xml`.

Web, security and core

Library	Version	What it does	Why DeployHub uses it	License
Spring Boot	4.1.2	Starts the app, wires everything together, embeds the web server, reads settings	The standard way to build Java web services; one setup for web, security, data and monitoring	Apache-2.0
Spring Framework (spring-core, spring-webmvc)	7.0.10	Dependency injection, transactions, REST controllers	Foundation of Spring Boot; REST endpoints such as /api/projects	Apache-2.0
Apache Tomcat (embedded)	11.0.27	The web server inside the backend (port 8080)	Comes with Spring Boot; no separate server to install	Apache-2.0
Spring Security	7.1.2	Login sessions, BCrypt passwords, CSRF protection, access rules	Proven security instead of hand-written login code; protects every /api endpoint	Apache-2.0
Hibernate Validator	9.1.5	Checks request data (e.g. project names, ports, commands)	Rules written next to the fields (@NotBlank, @Pattern); clear 400 errors	Apache-2.0
Jackson	2.21 / 3.1	Converts Java objects to JSON and back	All API requests and responses are JSON; also writes the generated Dockerfile's start command safely	Apache-2.0
Spring Boot Mail, Jakarta Mail (Eclipse Angus)	4.1.2 / 2.1.5 (Angus 2.0.5)	Sends email over SMTP: the 6-digit password-reset code (Forgot password)	The standard Java email API; Spring Boot sets it up from the MAIL_* settings in .env (Gmail by default)	Apache-2.0 / EPL-2.0, BSD-3
Logback	1.5.38	Writes the backend's own log	Spring Boot's default logger	EPL-1.0 / LGPL-2.1

Database access

Library	Version	What it does	Why DeployHub uses it	License
Spring Data JPA	4.1.2	Repositories: database queries from method names or short JPQL	Little code for many queries (e.g. findByProjectIdAndStatus)	Apache-2.0
Hibernate ORM	7.4.13	Maps Java classes (Project, Deployment, ...) to tables	Works the same on Oracle and H2; optimistic locking (@Version) for safe status changes	Apache-2.0
Flyway (+ Oracle support)	12.4.0	Applies the schema versions V1-V9 in order, once each	Every database (yours, the tests', a future server) gets exactly the same tables	Apache-2.0
HikariCP	7.0.2	Pool of database connections	Fast and small; the pool is kept tiny (5) for Oracle Free	Apache-2.0
Oracle JDBC (ojdbc11, oraclepki)	23.26.3	Driver to talk to Oracle; wallet support for Oracle Cloud	Needed for the oracle profile, today and for Oracle Autonomous Database later	Oracle FUTC (free use)
H2 Database	2.4.240	Small database inside the app (file or memory), in Oracle mode	Fast automated tests without Docker; the local profile	MPL-2.0 / EPL-1.0

Monitoring and API documentation

Library	Version	What it does	Why DeployHub uses it	License
Spring Boot Actuator	4.1.2	Health checks: /actuator/health/liveness and /readiness	Tells a load balancer when DeployHub is ready (Milestone 5)	Apache-2.0
Micrometer (+ Prometheus registry)	1.17.2	Counts and times things: queue depth, wait time, throughput	Metrics at /actuator/prometheus in the format monitoring tools read	Apache-2.0
springdoc-openapi (+ Swagger UI)	3.2.0 / 5.34	Describes the API and shows it as a web page	Browse and try every endpoint at /swagger-ui/index.html	Apache-2.0

Developer convenience and tests

These are used while building and testing, not in the running app (Lombok only at compile time).

Library	Version	What it does	Why DeployHub uses it	License
Lombok	1.18.48	Generates getters, constructors and loggers at compile time	Shorter entity and service classes	MIT
JUnit Jupiter	6.0.3	Runs the tests (232 on H2, 136 on Oracle, 7 Docker)	The standard Java test framework	EPL-2.0
Spring Boot Test, MockMvc	4.1.2	Starts the real app in tests and sends HTTP requests to it	Tests go through security, validation and the database like real requests	Apache-2.0
Mockito	5.23.0	Stand-ins (mocks) for parts of the system	Test the MySQL feature without MySQL; fake log writers	MIT
AssertJ, Hamcrest, JsonPath	3.27 / 3.0 / 2.10	Readable checks of results and JSON responses	Clear test failures	Apache-2.0, BSD-3
Testcontainers (Oracle Free module, Ryuk)	2.0.5	Starts a real Oracle in Docker for the oracle test suite, removes it afterwards	H2 alone can't prove Oracle compatibility	MIT

3. Frontend packages (Angular, managed by npm)

Declared in `deployhub-frontend/package.json`; exact versions locked in `package-lock.json`.

In the web pages (shipped to the browser)

Library	Version	What it does	Why DeployHub uses it	License
Angular (core, common, router, forms, platform-browser)	21.2.25	The framework: components, pages, navigation, forms, signals	Structured, typed single-page app; standalone components without NgModules	MIT
Angular CDK	21.2.14	Low-level UI helpers (overlays, accessibility)	Required by PrimeNG	MIT
PrimeNG	21.1.10	Ready-made UI components: buttons, tables, dialogs, selects, toasts	A complete, accessible component set instead of building each control	MIT
@primeuix/themes	2.0.3	PrimeNG's theme (Aura), customised to DeployHub's indigo colours	Consistent dark design	MIT
PrimeIcons	7.0.0	The icon font (pi pi-...)	Matches PrimeNG	MIT
RxJS	7.8.2	Streams for HTTP calls, polling, retries	Angular's HTTP client uses it; polling with automatic recovery	Apache-2.0
tslib	2.8.1	Small runtime helpers for compiled TypeScript	Required by Angular builds	0BSD

Build and test tools (not shipped)

Library	Version	What it does	Why DeployHub uses it	License
Angular CLI, @angular/build	21.2.28	ng serve, ng build, ng test; uses esbuild and Vite inside	Fast builds; the dev server proxies /api to the backend	MIT
Angular compiler / compiler-cli	21.2.25	Compiles templates and TypeScript	Required by Angular	MIT
TypeScript	5.9.3	Typed JavaScript	Catches mistakes before the browser does	Apache-2.0
Vitest	4.1.12	Runs the 98 frontend tests	Angular 21's default test runner; fast	MIT
jsdom	27.4.0	A browser imitation for tests	Frontend tests run without a real browser	MIT

Fonts

Font	Used for	Source	License
Inter	All text	Google Fonts (loaded by the browser from fonts.googleapis.com)	SIL OFL 1.1
JetBrains Mono	Code, logs, addresses	Google Fonts	SIL OFL 1.1

Note: because the fonts come from Google, a visitor's browser contacts Google when it opens DeployHub. They could be bundled with the app instead if that matters.

4. Docker images (programs DeployHub runs in containers)

Image	Used for	Why	License
gvenzl/oracle-free:23-slim-faststart	DeployHub's own database (Oracle Database Free 23.26)	Free Oracle, no cloud account; same engine as Oracle Cloud later	Oracle Free license; image Apache-2.0
mysql:8.4	Apps' one-click databases (one shared server)	The most common database for web apps; LTS release	GPL-2.0 (runs as a separate program)
nginx:1.27-alpine	Each project's fixed address and /api routes	Small (9 MB in memory), fast, reloads without downtime	BSD-2-Clause
cloudflare/cloudflared	Public links (quick tunnels)	Free https link without an account or opening ports	Apache-2.0
node:22-alpine	Building and running Node.js and static-site projects	Current LTS Node, small image	MIT
eclipse-temurin:21-jdk	Building and running Java/Spring Boot projects	Free, well-maintained Java 21 builds	GPL-2.0 + Classpath Exception
python:3.12-slim	Building and running Python projects	Official Python image	PSF
testcontainers/ryuk	Tests only: cleans up test containers	Comes with Testcontainers	MIT

5. Tools on your Mac

Tool	Version	Used for
Java (Oracle JDK)	21.0.9 LTS	Building and running the backend
Maven (via ./mvnw)	3.10.0	Building the backend and downloading its libraries
Node.js / npm	24.11.1 / 11.6.2	Building and running the frontend
Docker Desktop	4.85 (engine 29.6.2)	Oracle, MySQL, builds and app containers
git	2.39.5	Version control; DeployHub also uses it to fetch apps' code (read-only)

6. External services DeployHub contacts

Service	When	Direction
GitHub / GitLab / Bitbucket	Fetching an app's code when it is deployed	read-only download
Docker Hub	First use of an image (Node, Java, MySQL, nginx, ...)	download
npm registry, Maven Central, PyPI	Apps' builds download their packages	download
Cloudflare	Only for projects with a public link switched on	outgoing tunnel
Gmail (smtp.gmail.com)	Only when someone asks for a password-reset code, and only if MAIL_USERNAME is set in .env	outgoing email
Google Fonts	When a browser opens DeployHub's pages	download (by the browser)

DeployHub never pushes code, creates webhooks or changes settings on any of these services.

7. Licenses in one sentence

- Almost everything is under **permissive open-source licenses** (Apache-2.0, MIT, BSD, EPL, OFL): free to use, also commercially, keeping their copyright notices.
- **Oracle JDBC** (Oracle Free Use Terms) and **Oracle Database Free** are free to use, also in production, within Oracle Free's limits.
- **MySQL** and **Temurin** are GPL-licensed but run as separate programs in containers; DeployHub's code doesn't include them.
- Nothing here costs money.

8. What DeployHub saves: manual deployment vs DeployHub

Deploying an app by hand typically takes an engineer **2 to 3 days** the first time. These libraries and tools are what let DeployHub do the repetitive part automatically:

Task	Manual (engineer)	With DeployHub
Fetch the code, install dependencies, build	1-3 hours (first time, fixing environment problems)	**Automatic**, a few minutes
Write a Dockerfile / server setup for the app	half a day to a day	**Generated automatically** per app type
Create a database, user and password, connect the app	1-3 hours	**One click** (Add MySQL database)
Store secrets safely (passwords, JWT secret)	hours (often done unsafely)	**Encrypted** environment variables
Reverse proxy (nginx), link frontend and backend, CORS	half a day to a day	**Automatic** address plus /api route
Check the app really works before switching	often skipped	**Health check** before going LIVE
Switch to the new version without downtime	hours, and risky	**Automatic**
Roll back a bad release	stressful hours	**One click**, in minutes
Logs, status, history of every deployment	hand-made	**Live logs**, history, metrics
Public link	hours (domain, DNS, firewall)	**One click** (temporary today; permanent after Milestone 7)
Every later deployment	**hours each time**	**One click, a few minutes**

Proof from this project: the Employee Management app went from code on GitHub to a live, shareable app with database, backend and frontend in one afternoon. Most of that time went into fixing problems *in the app itself* (its setup script, a wrong sample password), which are now fixed for good.

What DeployHub does not do (yet)

- **The cloud server setup** (Milestone 7): needed once, then it serves all projects.
- **A permanent domain with HTTPS**: after Milestone 7.
- **Database backups and monitoring of live apps**: planned.
- **Running an app's own tests before deploying**, and **deploying on every push** (webhooks): can be added.
- **Bugs inside an app**: DeployHub shows the error clearly, but can't fix the app's code.
- **Big or unusual apps** (many services, special hardware, heavy traffic) still need an engineer's design work.

Realistic summary

Situation	Manual	With DeployHub
First deployment of a new app	2-3 days	about 30-60 minutes (settings once, plus fixing the app's own problems if it has any)
Every deployment after that	hours	one click, a few minutes, with one-click rollback
After Milestone 7		the same, plus a permanent public address for any number of users and apps

This is the same promise as Render, Railway and Vercel, which DeployHub is modelled on.

Full lists: `./mvnw dependency:list` (backend, 186 artifacts) and `npm ls --all` (frontend, ~480 packages).

Libraries and their purpose (update)

The current list from pom.xml and package.json · 29 October 2026. Replaces the list on the earlier pages.

Backend (Java 21, Spring Boot 4.1)

Library	Purpose
spring-boot-starter-webmvc	Web server and REST API; streaming live logs and database exports
spring-boot-starter-security	Login sessions, password checks, CSRF protection, access rules
spring-boot-starter-data-jpa (Hibernate)	Reading and writing the database with Java classes
spring-boot-starter-validation	Input checks: email, letters-only names, passwords, URLs
spring-boot-starter-mail	OTP and password-reset emails through Gmail
spring-boot-starter-flyway, flyway-database-oracle	Database changes V1 to V15, the same on every database
spring-boot-starter-actuator	Health checks, liveness and readiness
micrometer-registry-prometheus	Metrics (queue, build times, retries) for Prometheus
springdoc-openapi-starter-webmvc-ui	API documentation page (Swagger UI)
anthropic-java (3.2.0)	Claude API, when the assistant uses Claude. Gemini uses Java's built-in HTTP client
ojdbc11, oraclepki	Oracle database driver; Oracle wallet for the cloud Autonomous Database
h2, spring-boot-h2console	In-memory database for fast tests; its browser console (local only)
lombok	Generates getters, setters and constructors

Backend tests and build

Library	Purpose
spring-boot-starter-*-test	JUnit 5, Mockito, AssertJ, MockMvc (378 backend tests)
testcontainers (junit-jupiter, oracle-free), spring-boot-testcontainers	Real Oracle in Docker for the Oracle tests
maven-compiler / surefire / spring-boot-maven-plugin	Compile (with Lombok), run test groups, run and package the app

Frontend (Angular 21)

Library	Purpose
@angular/core, common, platform-browser	The Angular framework: components, signals, browser
@angular/router	Pages and route guards
@angular/forms	Sign-up, login, project form, chat input, validation
@angular/cdk	Building blocks used by PrimeNG (overlays, accessibility)
primeng, @primeuix/themes, primeicons	UI components, the theme system (orange light/dark on Aura), icons
rxjs	Backend calls and polling deployment status
tslib	TypeScript run-time helpers
@angular/cli, build, compiler-cli, typescript	Run, build and compile the frontend
vitest, jsdom	Frontend tests (164) in a simulated browser

Not libraries but services and tools DeployHub runs: Docker, nginx, cloudflared, MySQL, Razorpay Checkout, Google Gemini API. Java built-ins handle HTTP calls (Razorpay, Gemini), AES-GCM encryption, HMAC and photo resizing, without extra libraries.